CLASE 30

Django Templates

Separando HTML del código Python

Hoy aprenderán:

- Sistema de templates de Django
- Variables dinámicas {{ variable }}
- Tags de control {% if %}, {% for %}
- Template inheritance (extends, block)
- Static files (CSS, imágenes, JS)

Objetivo:

Construir la Landing Page del Dashboard E-commerce

El Problema Actual

HTML en strings de Python:

- HTML mezclado con Python = difícil de leer
- No hay syntax highlighting para el HTML
- Imposible trabajar con diseñadores
- No se puede agregar CSS fácilmente
- Un pequeño cambio visual = editar Python

La solución: Templates

Python (views.py) → **HTML (templates/)**

Lógica y datos — Presentación

- Lógica separada de presentación
- Syntax highlighting funciona
- Diseñadores editan HTML directo
- CSS y JS en archivos propios

Templates = separar lo que se ve de lo que se calcula

Configurar Templates

Paso 1: Crear carpetas de templates

Una carpeta por cada app del proyecto.
Así cada app tiene sus propios templates organizados.

Paso 2: Configurar settings.py

Decirle a Django dónde buscar los templates.
En la variable TEMPLATES → DIRS.

Estructura de carpetas:

```
templates/
  core/
    home.html, about.html, contact.html
  productos/
    lista.html, detalle.html
  dashboard/
    panel.html
```

Una carpeta por app = todo organizado. Como tener una carpeta para cada materia en la escuela.

render() — Conectar View con Template

ANTES:

HttpResponse con HTML en strings.
Todo mezclado, imposible de mantener.

AHORA:

render() conecta la vista con un template.
HTML en su archivo, Python en el suyo.

¿Cómo funciona render()?

request
Info del usuario

+

`'core/home.html'`
Qué template usar

+

context
Datos para el HTML

render() es como un cartero: le das la dirección (template) y el paquete (context), él lo entrega al navegador

Variables: {{ variable }}

{{ }} son como huecos en un formulario. El formulario (template) tiene los espacios, y Python llena los datos.

Vista (views.py):

Crea un diccionario context con los datos que necesita el template.

Template (home.html):

Usa {{ nombre_variable }} para mostrar esos datos en el HTML.

Ejemplo:

```
context = {'titulo': 'Dashboard', 'usuario': 'Juan'}

<h1>{{ titulo }}</h1>  →  <h1>Dashboard</h1>

<p>{{ usuario }}</p>   →  <p>Juan</p>
```

También accedes a atributos con punto: {{ producto.nombre }}

Filtros: Transformar Datos

Filtros = Instagram para datos. Tomas el dato y le aplicas un efecto.

upper

```
{{ nombre|upper }}  →  JUAN
```

lower

```
{{ nombre|lower }}  →  juan
```

truncatewords:N

```
{{ texto|truncatewords:10 }}  →  Primeras 10 palabras...
```

floatformat:2

```
{{ precio|floatformat:2 }}  →  29.99
```

date:"d/m/Y"

```
{{ fecha|date:"d/m/Y" }}  →  15/03/2024
```

length

```
{{ lista|length }}  →  5
```

default:"valor"

```
{{ bio|default:"Sin bio" }}  →  Sin bio
```

title

```
{{ nombre|title }}  →  Juan Pérez
```

Sintaxis: `{{ variable|filtro }}` o `{{ variable|filtro:argumento }}`

Tags: {% if %} — Condicionales

{% %} son instrucciones para el template. if funciona como en Python pero con sintaxis Django.

Ejemplo básico:

```
{% if usuario %}
  <p>Hola, {{ usuario }}!</p>
{% else %}
  <p>Hola, visitante!</p>
{% endif %}
```

Con elif — indicador de stock:

```
{% if stock > 10 %}
  <span class="verde">Disponible</span>
{% elif stock > 0 %}
  <span class="amarillo">Últimas ud.</span>
{% else %}
  <span class="rojo">Agotado</span>
{% endif %}
```

Siempre cerrar con {% endif %} — Django no usa indentación para cerrar bloques

Tags: {% for %} — Loops

Recorrer una lista de productos:

```
{% for producto in productos %}
  <li>{{ producto.nombre }} - ${{ producto.precio }}</li>
{% empty %}
  <li>No hay productos</li>
{% endfor %}
```

Variables especiales:

`forloop.counter`
Número (1, 2, 3...)

`forloop.counter0`
Desde 0 (0, 1, 2...)

`forloop.first`
True si es el primero

`forloop.last`
True si es el último

{% empty %} se muestra cuando la lista está vacía. Es como un plan B automático.

Uso real: tablas de datos, listas de productos, menús de navegación, tarjetas de dashboard

EJERCICIO RÁPIDO

Crear template: dashboard/panel.html

1. Saludo al usuario (mostrar nombre si existe, si no "visitante")
2. Lista de ventas recientes usando {% for %}
3. Indicador de estado según monto (> $500 verde, > $100 amarillo, sino rojo)
4. Usar forloop.counter para numerar las ventas

Tiempo: 8 minutos

Implementémoslo juntos

Template Inheritance: extends y block

extends = "Copio todo de base.html y solo cambio estas partes." Igual que herencia en Python.

base.html (padre):

```
<head> {% block title %} </head>
<nav> Menú común </nav>

{% block content %}
{% endblock %}

<footer> Pie común </footer>
```

home.html (hijo):

```
{% extends 'base.html' %}

{% block title %}Inicio{% endblock %}

{% block content %}
  <h1>Bienvenido</h1>
{% endblock %}
```

Ventaja: cambiar el nav en base.html actualiza TODAS las páginas automáticamente. Principio DRY.

BREAK

10 minutos

Volvemos con:

Static files (CSS, imágenes) y Landing Page completa

Static Files: CSS, JS, Imágenes

Configurar settings.py:

```
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

Estructura:

```
static/
  css/styles.css
  js/main.js
  img/logo.png
```

Cómo usar en un template:

```
Paso 1:  {% load static %}                    ← Al inicio del template

Paso 2:  <link href="{% static 'css/styles.css' %}" rel="stylesheet">

Paso 3:  <img src="{% static 'img/logo.png' %}" alt="Logo">
```

NUNCA hardcodear rutas: /static/css/styles.css ✘

Siempre usar: {% static 'css/styles.css' %} ✔

Usando Static Files en Templates

CSS:

```
{% load static %}
<link href="{% static 'css/styles.css' %}" rel="stylesheet">
```

JavaScript:

```
<script src="{% static 'js/main.js' %}"></script>
```

Imágenes:

```
<img src="{% static 'img/logo.png' %}" alt="Logo">
```

Si no ven los cambios de CSS: Ctrl+Shift+R (hard refresh). El navegador cachea estáticos.

include — Componentes Reutilizables

include es como copiar-pegar un pedazo de HTML. Pero solo lo escribes una vez.

Básico:

```
{% include 'partials/_navbar.html' %}
{% include 'partials/_footer.html' %}
```

Con parámetros:

```
{% include 'partials/_card.html'
   with titulo="Ventas" valor=total %}
```

Convención: _ al inicio = parcial

templates/partials/

- _navbar.html
- _footer.html
- _card.html
- _stat_card.html

extends vs include:

extends → Estructura completa (layout de la página)

include → Fragmentos pequeños (cards, alerts, nav)

PROYECTO: Landing Page Completa

Secciones de la Landing:

1. Hero Section — Título grande, descripción, botón CTA
2. Features — 3-4 tarjetas con características del producto
3. Stats — Estadísticas del negocio con números grandes
4. CTA Final — Llamada a acción con botón

Requisitos:

- Heredar de base.html
- Usar CSS de styles.css
- Datos dinámicos desde la vista
- Al menos un {% for %}
- Al menos un {% if %}
- Usar {% include %} para cards

Tiempo: 20 minutos

CSS para la Landing Page

Hero:	Fondo oscuro, texto centrado, botón CTA grande
Features Grid:	CSS Grid 4 columnas, cards con sombra
Stats:	Flexbox, números grandes 48px, fondo claro
CTA:	Fondo teal, botón blanco, texto centrado
Responsive:	Media queries para móvil (2 cols → 1 col)

El CSS completo lo codificamos juntos paso a paso en VS Code

🔗 {% url %} — Enlaces Dinámicos

Hardcoded (MAL) ✘

```
<a href="/productos/5/">Ver</a>
```

Si cambias la URL en urls.py, todos los enlaces se rompen.

Dinámico (BIEN) ✔

```
{% url 'productos:detalle' 5 %}
```

Django calcula la URL. Si cambia en urls.py, se actualiza sola.

Ejemplos:

`{% url 'core:home' %}`	→	**/**
`{% url 'productos:lista' %}`	→	**/productos/**
`{% url 'productos:detalle' 5 %}`	→	**/productos/5/**
`{% url 'dashboard:panel' %}`	→	**/dashboard/**

💡 El nombre viene de los name= que pusimos en urls.py de la clase 29

Resumen — Django Templates

- Sistema de templates configurado
- Variables {{ }} y filtros
- Tags {% if %}, {% for %}
- Template inheritance (extends, block)
- Static files (CSS)
- Componentes con include
- Enlaces dinámicos con {% url %}
- Landing page con diseño real

PRÓXIMA CLASE: Models

- Definir estructura de datos con modelos
- ORM de Django (Python en lugar de SQL)
- Migrations (versionar la base de datos)
- Admin automático de Django